

PART 1 — Setup

Step 1: Start MongoDB server

```
mongod  
(or start MongoDB service)
```

Step 2: Open Mongo Shell

```
mongosh
```

Step 3: Create database & collection (mongosh)

```
use bankDB
```

```
db.accounts.insertMany([  
  { _id: 101, customerName: "Arun",  accountType: "Savings",  
    balance: 50000, branch: "Chennai" },  
  { _id: 102, customerName: "Divya", accountType: "Current",  
    balance: 120000, branch: "Bangalore" },  
  { _id: 103, customerName: "Rahul", accountType: "Savings",  
    balance: 30000, branch: "Hyderabad" }  
])
```

Verify:

```
db.accounts.find()
```

PART 2 — Export Data (mongoexport)

Step 4: Exit mongosh

```
exit - press ctrl+c
```

Step 5: Export to JSON

```
mongoexport --db bankDB --collection accounts --out
accounts.json
```

 Output file: accounts.json

Step 6: Export to CSV

```
mongoexport \
--db bankDB \
--collection accounts \
--type csv \
--fields _id,customerName,accountType,balance,branch \
--out accounts.csv
```

 Output file: accounts.csv

PART 3 — Import Data (mongoimport)

Step 7: Create new database for import

```
mongosh
use bankImportDB
exit
```

Step 8: Import JSON

```
mongoimport --db bankImportDB --collection accounts --file
accounts.json --jsonArray
```

Step 9: Import CSV

```
mongoimport \
--db bankImportDB \
--collection accountsCSV \
--type csv \
--headerline \
--file accounts.csv
```

Step 10: Verify imported data

```
mongosh
use bankImportDB
db.accounts.find()
db.accountsCSV.find()
```

PART 4 — Using VS Code / Any IDE

Step 11: Open terminal inside IDE

Example (VS Code):

Terminal → New Terminal
Run:

```
mongoexport --db bankDB --collection accounts --out
accounts.json
mongoimport --db testDB --collection accounts --file
accounts.json --jsonArray
```

STEP 1: Start MongoDB without authentication (first time only)

Open terminal and run:

```
mongod --dbpath /data/db
```

Then open another terminal:

```
mongosh
```

👉 Right now, MongoDB has **no security**. We will create an admin user.

STEP 2: Switch to **admin** database

```
use admin
```

MongoDB stores admin users in the **admin** database .

STEP 3: Create an Admin User

```
db.createUser({
  user: "adminUser",
  pwd: "admin123",
  roles: [ { role: "userAdminAnyDatabase", db: "admin" } ]
})
```

✅ Output:

```
{ ok: 1 }
```

Meaning: user created successfully.

STEP 4: Stop MongoDB and restart with authentication enabled

Stop server (Ctrl + C), then restart:

```
mongod --auth --dbpath /data/db
```

Now authentication is ON 🔒

STEP 5: Login as Admin User

Open shell:

```
mongosh -u adminUser -p --authenticationDatabase admin
```

Enter password when asked:

```
admin123
```

✅ Now you are logged in as admin.

Verify:

```
db.runCommand({ connectionStatus: 1 })
```

Output:

```
{
  authInfo: {
    authenticatedUsers: [ { user: "adminUser", db:
"admin" } ],
```

```
    authenticatedUserRoles: [ ... ]
  },
  ok: 1
}
```

Create Normal Database User

STEP 6: Create Application Database

```
use schoolDB
```

STEP 7: Create User for that Database

```
db.createUser({
  user: "schoolUser",
  pwd: "school123",
  roles: [ { role: "readWrite", db: "schoolDB" } ]
})
```

This user:

✓ can read/write only schoolDB

✗ cannot access other DBs

```
db.createUser({
... user: "firstUser",
... pwd: "first123",
... roles: [{role:"readWrite",
db:"Day3"}]
... })
```

STEP 8: Login as Normal User

```
mongosh -u schoolUser -p --authenticationDatabase schoolDB
Enter password:
```

```
school123
```

Test:

```
show dbs
```

👉 You will only see `schoolDB` (and maybe `test`).

Useful Authentication Commands

Check logged-in user

```
db.runCommand({ connectionStatus: 1 })
```

List users in current DB

```
show users  
or
```

```
db.getUsers()
```

Change password

```
db.changeUserPassword("schoolUser", "newpass123")
```

Delete user

```
db.dropUser("schoolUser")
```

Grant extra role

```
db.grantRolesToUser(  
    "schoolUser",  
    [ { role: "dbAdmin", db: "schoolDB" } ]  
)
```

1. MongoDB Atlas – Overview

MongoDB Atlas = Cloud-managed MongoDB service

You DON'T need to:

❌ install MongoDB

❌ manage backups

- ✗ configure replication
- ✗ handle scaling

Atlas gives:

- ✓ Managed clusters
- ✓ High availability (replica sets)
- ✓ Auto backups
- ✓ Monitoring & alerts
- ✓ Security (users, IP whitelist)
- ✓ Global regions

Works on:

- ☁ AWS
- ☁ Azure
- ☁ Google Cloud

2. Cluster Types in Atlas

◆ (A) Shared Cluster – Free / Low cost

Example: **M0, M2, M5**

Used for:

- ✓ Learning
- ✓ Demo
- ✓ Small apps

Limits:

- ✗ Limited RAM & storage
- ✗ No advanced performance tuning

◆ (B) Dedicated Cluster – Production

Example: **M10, M20, M30, M40...**

Used for:

- ✓ Production apps
- ✓ High traffic
- ✓ Enterprise systems

◆ (C) Serverless Instance

Used for:

- ✓ Variable workloads
- ✓ Event-driven apps

You pay:

💰 Per read/write operation

3. Regions in Atlas (Very Important)

Atlas lets you choose:

- ✓ Cloud provider
- ✓ Geographic region

Example regions:

AWS → Mumbai, Singapore, Frankfurt, US-East

Azure → Central India, West Europe

GCP → Asia-South1, US-Central

Why region matters:

- ✓ Lower latency
- ✓ Legal compliance
- ✓ Faster queries
- ✓ Disaster recovery

👉 If your users are in India → choose **Mumbai / Central India**

4. Step-by-Step: Create Atlas Account & Cluster

STEP 1: Create Atlas Account

Go to:

👉 <https://www.mongodb.com/cloud/atlas>

Sign up using:

- ✓ Google
- ✓ Email

STEP 2: Create Project

Atlas groups clusters inside **Projects**

Click:

➡ **New Project**

Name: TrainingProject

STEP 3: Build a Cluster

Click:

➡ **Build a Cluster**

Choose:

Cluster Type:

✓ Shared → M0 (Free)

Cloud Provider:

✓ AWS (or Azure/GCP)

Region:

✓ Mumbai / nearest region

Cluster Name:

myFirstCluster

Click:

➡ **Create Cluster**

(wait 2–3 minutes)

5. Step-by-Step: Security Setup (Very Important)

STEP 4: Create Database User

Go to:

➡ Security → Database Access → Add New User

Fill:

Username: atlasUser

Password: atlas123
Role: readWriteAnyDatabase
Click:
➡ Add User

STEP 5: Allow Network Access (IP Whitelist)

Go to:
➡ Security → Network Access → Add IP Address

Choose:
✔ Allow Access from Anywhere (0.0.0.0/0)
(or your own IP for security)

6. Step-by-Step: Connect to Cluster

Click:
➡ Connect
➡ Connect with MongoDB Shell

Copy command like:

```
mongosh "mongodb+srv://myfirstcluster.xxxxx.mongodb.net" -u  
atlasUser
```

Enter password when prompted.

You are now connected 🎉

7. Step-by-Step: Work with Data

Create Database & Collection

```
use schoolDB
```

```
db.students.insertOne({  
  name: "Ravi",  
  age: 22,  
  course: "MongoDB"  
})
```

Read Data

```
db.students.find()
```

Update Data

```
db.students.updateOne(  
  { name: "Ravi" },  
  { $set: { age: 23 } }  
)
```

Delete Data

```
db.students.deleteOne({ name: "Ravi" })
```

1. Scaling in MongoDB Atlas

What is Scaling?

Scaling = increasing or decreasing cluster capacity when load changes.

Atlas supports:

Vertical Scaling → change cluster tier (M10 → M20)

Horizontal Scaling → add shards (for big data)

Vertical Scaling (Resize Tier)

Example:

M10 → M20 (more RAM, CPU, storage)

Steps:

Login to **MongoDB Atlas**

Open your **Project**

Click your **Cluster name**

Click **Configuration** (or **Edit Configuration**)

You will see:

Cluster Tier (M0, M10, M20...)

Storage size

Cloud provider

Region

Change:

Cluster Tier: M10 → M20
(or any higher/lower tier)

Click:

➡ **Review Changes**

➡ **Apply Changes**



Atlas performs a rolling update (no major downtime).

Horizontal Scaling (Sharding)

Used when:

Data is huge (TBs)

Queries are heavy

Steps:

Go to:

➡ Cluster → **Configuration**

Enable:

➡ **Sharding**

Choose:

Number of shards: 2 or more

Shard key: { `userId: 1` } (example)

Click:

➡ **Apply Changes**

Now data is distributed across shards.

2. Resizing in MongoDB Atlas

What is Resizing?

Resizing = changing:

- ✓ RAM
- ✓ CPU
- ✓ Storage
- ✓ Tier

It is basically **vertical scaling**.

Resize Cluster (Step-by-step)

Go to **Atlas Dashboard**

Select your **Cluster**

Click:

- ➔ **Configuration**
- ➔ **Edit Configuration**

Change any of:

Cluster tier (M20 → M10 or M10 → M30)

Disk size (20GB → 40GB)

Click:

- ➔ **Review Changes**
- ➔ **Apply Changes**

Atlas shows:

- ✓ estimated cost
- ✓ downtime (if any)

Important Notes:

You can **scale up anytime**

Scaling down may need:

Less data than new storage limit

M0 (free) → can upgrade to M2/M5

Dedicated clusters support full resizing

3. Pause and Resume Cluster

What is Pause?

Pause = stop cluster temporarily

- ✓ Saves money
- ✓ Keeps data
- ✓ Used for training/dev

⚠ Only for:

M0, M2, M5

Non-production mostly

Pause Cluster (Step-by-step)

Go to **Atlas Dashboard**

Click your **Cluster**

Click:

➡ **Pause**

Confirm:

➡ **Pause Cluster**

Cluster state becomes:

Paused

You cannot connect while paused.

Resume Cluster (Step-by-step)

Go to **Atlas Dashboard**

Click your paused cluster

Click:

➡ **Resume**

Wait 2–3 minutes

Status becomes:

Available

Data is still there ✓

Real-Time Example

Training Environment:

Cluster: M0

Used during class

After class → Pause

Next day → Resume

Production App:

Cluster: M20

Traffic increases → Resize to M30

Later traffic drops → Resize to M20

1. Users (Database Users in Atlas)

What is a User?

A **database user** is used by applications or shell to:

- ✓ login
- ✓ access databases
- ✓ run queries

Step-by-Step: Create a Database User

1. Login to **MongoDB Atlas**
2. Open your **Project**
3. Go to:
 ➡ **Security** → **Database Access**
4. Click:
 ➡ **Add New Database User**

Fill:

Authentication Method: Password

Username: appUser

Password: app123

5. Under **Database User Privileges**:
Choose role:

`readWriteAnyDatabase`
(or custom role)

6. Click:
➡ **Add User**

 User created successfully

Step-by-Step: View Users

Go to:
➡ **Security** → **Database Access**

You will see list of users:

`appUser`
`adminUser`

Step-by-Step: Delete User

1. Click user name
2. Click:
➡ **Delete**

2. Roles (Permissions)

What is a Role?

A role defines:
✓ what user can do
✓ on which database

Example roles:

- `read`
- `readWrite`
- `dbAdmin`

- `readWriteAnyDatabase`
- `atlasAdmin`

Step-by-Step: Assign Role to User

1. Go to:
➡ **Security** → **Database Access**
2. Click a user
3. Click:
➡ **Edit**
4. Under **Built-in Roles** choose:

Role: `readWrite`

Database: `schoolDB`

5. Click:
➡ **Update User**

Step-by-Step: Create Custom Role

1. Go to:
➡ **Security** → **Database Access**
2. Click:
➡ **Add Custom Role**

Example:

Role Name: `reportRole`

Database: `salesDB`

Actions: `find`, `aggregate`

3. Click:
➡ **Create Role**

Now assign it to user.

3. IP Whitelisting (Network Access)

What is IP Whitelisting?

It controls:

✓ which IPs can connect

✗ blocks all others

No IP = No connection (even with correct user/password)

Step-by-Step: Add IP Address

1. Go to:
➡ **Security** → **Network Access**
2. Click:
➡ **Add IP Address**

Choose one:

Option A: Allow your current IP

➡ **Add Current IP Address**

Option B: Allow all (not secure)

➡ **Allow Access from Anywhere**

0.0.0.0/0

3. Click:
➡ **Confirm**

Status becomes:

Active

Step-by-Step: Remove IP

1. Go to:
➡ **Network Access**
2. Click:
➡ **Delete** next to IP

4. Connection Strings

What is a Connection String?

It is the **URL used to connect**:

- Mongo shell
- Node.js
- Java
- Python
- Compass

Format (Atlas):

```
mongodb+srv://  
username:password@clustername.xxxxx.mongodb.net/dbname
```

Step-by-Step: Get Connection String

1. Go to **Clusters**
2. Click:
➡ **Connect**
3. Choose:
➡ **Connect with MongoDB Shell**
(or Compass / Drivers)

You will see:

```
mongosh "mongodb+srv://mycluster.xxxx.mongodb.net" -u appUser  
or URI:
```

```
mongodb+srv://appUser:<password>@mycluster.xxxx.mongodb.net/  
schoolDB  
Replace:
```

```
<password> → app123  
Final:
```

```
mongodb+srv://appUser:app123@mycluster.xxxx.mongodb.net/  
schoolDB
```

Test Connection (Shell)

```
mongosh "mongodb+srv://mycluster.xxxx.mongodb.net" -u appUser
Enter password.
```

Then:

```
show dbs
```

1. Automated Backups in MongoDB Atlas

What are Automated Backups?

Atlas automatically:

- ✓ takes backups
- ✓ stores them safely
- ✓ allows point-in-time restore
- ✓ works without manual commands

Available for:

- M10 and above (Dedicated clusters)
- Optional for M2/M5

Step-by-Step: Enable Automated Backups

1. Login to **MongoDB Atlas**
2. Open your **Project**
3. Click your **Cluster**
4. Go to:
➡ **Backup** (or **Backup** → **Configuration**)
5. Toggle:

Enable Backup = ON

6. Configure:

Backup Frequency: Continuous / Hourly

Retention: 7 days / 14 days / 30 days

7. Click:
➡ **Save**

 Automated backup is now enabled

Step-by-Step: View Backup Snapshots

1. Go to:
➡ **Cluster** → **Backup**
2. Click:
➡ **Snapshots**

You will see:

Daily snapshots
Timestamp
Size

2. Restore in MongoDB Atlas

Restore Types:

1. Snapshot Restore
2. Point-in-Time Restore

A) Snapshot Restore (to previous state)

1. Go to:
➡ **Cluster** → **Backup** → **Snapshots**
2. Select a snapshot
3. Click:
➡ **Restore**

Choose:

✓ Restore to same cluster

OR

✓ Restore to new cluster

4. Click:
➡ **Confirm Restore**

Atlas restores data from backup 

B) Point-in-Time Restore

Used when:

- ✓ data deleted accidentally
- ✓ want state at exact time (10:30 AM)

Steps:

1. Go to:
➡ **Backup** → **Restore**
2. Choose:
➡ **Point in Time Restore**
3. Select:

Date: 28 Jan

Time: 10:30 AM

4. Choose:
✓ Target cluster
5. Click:
➡ **Restore**

3. Metrics & Dashboards

What are Metrics?

They show:

- ✓ CPU usage
- ✓ Memory
- ✓ Disk I/O
- ✓ Connections
- ✓ Query performance

Step-by-Step: View Metrics

1. Go to **Atlas**
2. Open your **Cluster**
3. Click:
➡ **Metrics**

You will see graphs:

- CPU %
- Disk IOPS
- Network
- Connections
- Opcounters (insert/update/find)

You can select:

Last 1 hour

Last 6 hours

Last 24 hours

Step-by-Step: Use Performance Advisor

1. Cluster → **Performance Advisor**
2. It shows:
 - ✓ slow queries
 - ✓ missing indexes
 - ✓ recommendations

Click:

➡ **Create Index** (directly from UI)

4. Alerts in MongoDB Atlas

What are Alerts?

Alerts notify you when:

- ✓ CPU is high
- ✓ disk is full
- ✓ cluster down
- ✓ connections spike

Alert channels:

 Email

 SMS

 UI notifications

Step-by-Step: Create an Alert

1. Go to:
➡ **Alerts**
2. Click:
➡ **Add New Alert**

Choose condition:

Metric: CPU Usage

Condition: Above

Threshold: 75%

For: 5 minutes

Choose notification:

✓ Email

✓ Atlas UI

3. Click:
➡ **Save**

Step-by-Step: Common Alert Types

You can create alerts for:

- CPU > 80%
- Disk usage > 90%
- Too many connections
- Cluster unavailable
- Backup failed

Index Tuning & Slow Queries

Goal

Make queries faster by:

- ✓ finding slow queries
- ✓ creating proper indexes
- ✓ verifying index usage

Step 1: Identify Slow Queries (Atlas)

Atlas UI:

➡ Cluster → **Performance Advisor**

➡ Shows:

- slow queries
- missing indexes
- recommended indexes

Step 2: Identify Slow Queries (Shell)

```
db.students.find({ age: 25 }).explain("executionStats")
```

Look for:

COLLSCAN  (bad – full collection scan)

IXSCAN  (good – index used)

Step 3: Create Index

```
db.students.createIndex({ age: 1 })
```

Compound index:

```
db.students.createIndex({ course: 1, age: -1 })
```

Step 4: Verify Index

```
db.students.getIndexes()
```

Test again:

```
db.students.find({ age: 25 }).explain("executionStats")
```

Now it should show:

IXSCAN

Step 5: Remove Unused Index

```
db.students.dropIndex({ age: 1 })
```



Best Practices

- ✓ Index fields used in WHERE / filter
- ✓ Use compound indexes for multiple filters
- ✓ Avoid too many indexes
- ✓ Use Performance Advisor regularly

2 Driver, Connection & CRUD (Node.js)

We use **MongoDB Node.js Driver**

Step 1: Install Driver

```
npm install mongodb
```

Step 2: Connect to MongoDB Atlas

```
const { MongoClient } = require("mongodb");

const uri = "mongodb+srv://user:pass@cluster.mongodb.net/schoolDB";

const client = new MongoClient(uri);

async function connectDB() {
  await client.connect();
  console.log("Connected to MongoDB");
}

connectDB();
```

Step 3: Get Database & Collection

```
const db = client.db("schoolDB");
const students = db.collection("students");
```

Step 4: CRUD Operations

CREATE

```
await students.insertOne({ name: "Ravi", age: 22 });
```

READ

```
await students.find({ age: 22 }).toArray();
```

UPDATE

```
await students.updateOne(  
  { name: "Ravi" },  
  { $set: { age: 23 } }  
);
```

DELETE

```
await students.deleteOne({ name: "Ravi" });
```

Schemas, Models, Validation, Middleware (Mongoose)

We use **Mongoose ODM**

Step 1: Install Mongoose

```
npm install mongoose
```

Step 2: Connect using Mongoose

```
const mongoose = require("mongoose");  
  
mongoose.connect("mongodb+srv://  
user:pass@cluster.mongodb.net/schoolDB")  
.then(() => console.log("Connected"))  
.catch(err => console.log(err));
```

Step 3: Create Schema

```
const studentSchema = new mongoose.Schema({
```

```

name: {
  type: String,
  required: true
},
age: {
  type: Number,
  min: 18,
  max: 60
},
course: {
  type: String,
  enum: ["MongoDB", "Python", "Java"]
},
email: {
  type: String,
  unique: true
}
});

```

Step 4: Create Model

```

const Student = mongoose.model("Student", studentSchema);
Model = MongoDB collection interface

```

Step 5: Validation (Auto)

```

await Student.create({
  name: "Ravi",
  age: 17,      // ✗ fails (min 18)
  course: "MongoDB"
});
Error:

```

ValidationError: age must be >= 18

Step 6: Middleware (Hooks)

Middleware runs:

- ✓ before save
- ✓ after save
- ✓ before delete

Example: `pre("save")`

```
studentSchema.pre("save", function(next) {  
  console.log("Before saving student");  
  next();  
});
```

Example: `post("save")`

```
studentSchema.post("save", function(doc) {  
  console.log("Saved:", doc.name);  
});
```

Step 7: CRUD using Model

Create

```
await Student.create({  
  name: "Anita",  
  age: 24,  
  course: "Python",  
  email: "anita@gmail.com"  
});
```

Read

```
await Student.find({ course: "Python" });
```

Update

```
await Student.updateOne(  
  { name: "Anita" },  
  { age: 25 }  
);
```

Delete

```
await Student.deleteOne({ name: "Anita" });
```

How These Topics Connect

Topic	Purpose
-------	---------

Index tuning	Makes queries fast
Slow query detection	Finds performance issues
Driver	Connects app to MongoDB
CRUD	Manipulates data
Schema	Structure of data
Model	Interface to DB
Validation	Enforces rules
Middleware	Runs logic automatically